

AGRICONNECT FARM-TO-MARKET PLATFORM

PHASE 9: INDEXING AND QUERY OPTIMIZATION REPORT

MySQL 8.0+ | Query plans, FULLTEXT search, and insert experiment

1. Introduction

This report documents Phase 9 tests of indexes on product and order queries, a FULLTEXT search on product names, and an insert-time comparison with and without a secondary index. Temporary tables were used for the insert experiment to avoid changing the main project data.

2. Product query: composite index

Query: `EXPLAIN SELECT * FROM products WHERE category = 'FRUIT' AND unit_price <= 50000;`

Metric	Before index	After index
Access type	ALL	range
Selected key	None	idx_products_cat_price
Estimated rows	7	2
Extra	Using where	Using index condition

Index created: `CREATE INDEX idx_products_cat_price ON products(category, unit_price);`

Interpretation: the optimizer changed from a full table scan (ALL) to range access using the composite index. Estimated rows decreased from 7 to 2. These are optimizer estimates, not a measured 3.5× runtime speed-up.

3. Order query: composite index

Query: `EXPLAIN SELECT * FROM orders WHERE buyer_id = 4 ORDER BY order_date DESC;`

Metric	Before index	After index
Access type	ALL	ref
Selected key	None (possible key buyer_id)	idx_orders_buyer_date
Estimated rows	4	4
Extra	Using where; Using filesort	None reported

Index created: `CREATE INDEX idx_orders_buyer_date ON orders(buyer_id, order_date DESC);`

Interpretation: access changed from ALL to ref, and the post-index plan no longer reported “Using filesort.” This is consistent with the composite index supporting the buyer filter and date ordering. Estimated rows remained 4; elapsed-time improvement was not measured.

4. FULLTEXT product-name search

Statements tested:

```
ALTER TABLE products ADD FULLTEXT ft_product_name (product_name);
SELECT * FROM products WHERE MATCH(product_name) AGAINST('matooke');
```

The search returned the Matooke record: product_id 2, category TUBER, unit bunch, unit_price 25,000.00. MySQL displayed warning 124 explaining that InnoDB rebuilt the table to add its internal FTS_DOC_ID column; the FULLTEXT index creation succeeded.

5. Insert-performance experiment

Two temporary tables were tested: one with only its primary key and another with a composite secondary index on (category, unit_price). The same 1,000 generated rows were inserted into each.

Test table	Rows inserted	Displayed time
p9_insert_before (no secondary index)	1,000	0.016 sec
p9_insert_after (composite index)	1,000	0.016 sec

Both inserts displayed the same elapsed time at Workbench's reported precision. This single small test showed no measurable timing difference. Larger datasets and repeated runs are needed for a more reliable comparison.

6. Summary and conclusion

Test	Observed result	Conclusion
Products	ALL → range; rows 7 → 2	Improved selected access path.
Orders	ALL → ref; no filesort reported	Index supports buyer lookup and ordering.

Test	Observed result	Conclusion
FULLTEXT	"matooke" returned Matooke	Product-name search worked.
Insert test	0.016 sec for both	No measurable difference in this test.

Overall, Phase 9 demonstrated improved query plans for the tested product and order queries and a successful FULLTEXT search. The recorded insert test showed the same displayed duration before and after indexing. These results do not prove a measured 2× or greater runtime improvement; a separate controlled benchmark would be needed if the assessment requires that threshold.

Appendix A: Screenshot evidence

Supporting screenshots are stored in Phase9_Indexing/. They are referenced by filename and are not embedded in this PDF.

Filename	Evidence captured
P9_1_products_before.png	Product EXPLAIN before index
P9_2_products_after.png	Product EXPLAIN after index
P9_3_orders_before.png	Order EXPLAIN before index
P9_4_orders_after.png	Order EXPLAIN after index
P9_5_fulltext_search.png	FULLTEXT search result
P9_6_insert_before.png	Insert test without secondary index
P9_7_insert_after.png	Insert test with composite index